

Advanced Machine Learning



Bogdan Alexe,

bogdan.alex@fmi.unibuc.ro

University of Bucharest, 2nd semester, 2025-2026

Assignment 1 – deadline next week

- 3 problems = 2.7 points + 0.3 points (ex-officio)
- deadline: in about 4 weeks time, Tuesday 5th of May 2026, 23:59
 - late submission policy: maximum 3 days allowed, -10% (= 0.3 points) for each day
 - upload a pdf written in a scientific editor (Word, Latex, LyX) containing your solution here: <https://tinyurl.com/AML-2026-ASSIGNMENT1>
 - *is mandatory that you write your solution with a scientific editor, otherwise your solution would not be taken into account*
 - you can insert drawings for your proofs
 - after uploading your solution you should get a confirmation email; if not send me an email.
- for every problem write clear explanations, proofs to justify your answer (if you write just some indications you will not get too many points)
- do not share/copy the solution with/from your colleagues: you + your colleague/s will get 0 points

Recap - Computational resources of learning

For learning we need 2 type of resources:

1. *Information = training data*
 - so far we analyzed how much training data (sample size) we need in order to learn
 - *sample complexity*
2. *Computation = runtime*
 - for how much time an algorithm (that implements learning) will run, once we have sufficiently many training examples
 - *computational complexity*
 - crucial when we need fast ML applications (driver surveillance, stock exchange trading, etc)
 - runtime = number of elementary instructions executed - arithmetic operations over real numbers - in an asymptotic sense (with respect to input size) of the algorithm, e.g. $O(n)$ – where n is the size of the input size

Recap - Input size parameter of learning

What should play the role of the input size parameter in learning?

- accuracy ε , confidence δ and another parameter n related to the size/complexity of $\mathcal{X}$, $\mathcal{H}$
 - how much computation we need in order to get accuracy ε with confidence δ
 - want to have *efficient learning* (give a formal definition): polynomial in $1/\varepsilon$, $1/\delta$ and n (some parameter related to the size/complexity of domain/hypothesis class: more complex hypothesis needs more computation time)
- we study the runtime in an asymptotic sense by defining a sequence of pairs $(\mathcal{X}_n, \mathcal{H}_n)_{n=1,2,\dots}$ and studying asymptotic complexity of learning $\mathcal{X}_n, \mathcal{H}_n$ as n grows to ∞

Formal definition of computational complexity

Definition

Given a function $f : (0,1)^2 \rightarrow \mathbb{N}$, a learning task $(\mathcal{Z}, \mathcal{H}, \ell)$, and a learning algorithm A , we say that A solves the learning task in time $O(f)$ if there exists some constant number c , such that for every probability distribution $\mathcal{D}$ over $\mathcal{Z}$, and input $\epsilon, \delta \in (0,1)$, when A has access to samples generated i.i.d by $\mathcal{D}$, we have that:

- A terminates after performing at most $c * f(\epsilon, \delta)$ operations;
- the output of A , denoted h_A , can be applied to predict the label of a new example while performing at most $c * f(\epsilon, \delta)$ operations;
- the output of A is probably approximately correct; namely, with probability of at least $1 - \delta$ (over the random samples A receives):

$$L_{\mathcal{D}}(h_A) \leq \min_h L_{\mathcal{D}}(h) + \epsilon$$

Formal definition of efficient PAC learning (Valiant 1984)

In 1984, Leslie Valiant defined efficient PAC learning: PAC learnability + require the number of examples and the runtime of the algorithm A (training + testing) to be polynomial in $1/\epsilon$, $1/\delta$, n .

Example:

- $\mathcal{U}_n = \{h: B^n \rightarrow \{0,1\}\}$ - the concept class formed by all subsets of B^n
- $|\mathcal{U}_n| = 2^{2^n}$ - finite, so is PAC learnable with $m_{\mathcal{H}}(\epsilon, \delta)$ in the order of m :

$$m \geq \left\lceil \frac{1}{\epsilon} \left(2^n \log(2) + \log\left(\frac{1}{\delta}\right) \right) \right\rceil$$

- $\text{VCdim}(\mathcal{U}_n) = 2^n$ is not polynomial in n
- sample complexity exponential in n , number of variables
- it is not efficient PAC-learnable in any practical sense (need polynomial sample complexity)

Recap - Examples

1. $\mathcal{H}_{\text{conj}}^n$ = class of conjunctions of at most n Boolean literals $x_1, \dots, x_n$
 - in the realizable case (PAC learning) the runtime of an algorithm that finds an ERM hypothesis is $O(m_{\mathcal{H}}(\varepsilon, \delta) * n)$, so is polynomial in $1/\varepsilon, 1/\delta, n$
 - in the agnostic (unrealizable) case: unless $P = NP$, there is no algorithm whose running time is polynomial in $m_{\mathcal{H}}(\varepsilon, \delta)$ and n that is guaranteed to find an ERM hypothesis for the class of Boolean conjunctions.

2. $\mathcal{H}_{\text{rec}}^n$ = the class of axis aligned rectangles in $\mathbf{R}^n$
 - in the realizable case (PAC learning) the runtime of an algorithm that finds an ERM hypothesis is $O(m_{\mathcal{H}}(\varepsilon, \delta) * n)$, so is polynomial in $1/\varepsilon, 1/\delta, n$ (the algorithm has to find the minimal and the maximal values among the positive instances in the training sequence).
 - in the agnostic case for every fixed n , $\text{ERM}_{\mathcal{H}}$ can be implemented in time which is polynomial in $1/\varepsilon, 1/\delta, n$ (measures the complexity of the $\mathcal{H}_{\text{rec}}^n$) therefore we have efficient learning
 - however, as a function of n the runtime of the algorithm implementing the $\text{ERM}_{\mathcal{H}}$ presented is exponential in n . It can be proved that there is no better algorithm (unless $P = NP$) than the one proposed.

Relation between consistency and PAC learning

- a learning rule is consistent if:
 - input: $\mathcal{H}$ and $S = (x_1, y_1), \dots, (x_m, y_m)$
 - output: $h \in \mathcal{H}$, h is an ERM hypothesis, i.e. $h(x_i) = y_i$ (it perfectly fits the data)
- we know from the fundamental theorem of statistical learning that if $\text{VCdim}(\mathcal{H}) \leq d$ then $\mathcal{H}$ is PAC learnable (in the realizable case) by any consistent rule with sample complexity (given by the fundamental theorem):

$$C_1 \frac{d + \log(1/\delta)}{\epsilon} \leq m_{\mathcal{H}}(\epsilon, \delta) \leq C_2 \frac{d \log(1/\epsilon) + \log(1/\delta)}{\epsilon}$$

- theory: finite VC dimension $\implies$ PAC learning
- algorithmic strategy: finite VC dimension + consistency $\implies$ PAC learning
- if d is polynomial in n and the consistent rule has runtime polynomial in the sample size $m_{\mathcal{H}}(\epsilon, \delta)$ then we have efficient PAC learning
- so, efficient ‘consistent-hypothesis-finder’ $\implies$ efficient PAC learning
- does the converse implication holds?
 - yes, based on randomized algorithms

Today's lecture: Overview

- Randomized algorithms
- Intractability of learning
- Improper learning

Randomized algorithms

- a randomized algorithm A is allowed to use random numbers as part of its input
- the output of the algorithm A depends on the input, so it depends on the particular sequence produced by a random number generator
- we can speak of the probability that A has a given outcome
- a randomized algorithm A ‘solves’ a binary decision problem (YES/NO) if it behaves in the following way:
 - the algorithm always halts and produces an output (in polynomial time)
 - if the output of A is YES then the answer is 100% right
 - if the output of A is NO then the answer is right with probability $\geq 50\%$
- randomized algorithm for the problem “number n is composite (is not prime)”
 - if the output is YES then we are sure that number n is composite (not prime)
 - if the output is NO then with probability $\geq 50\%$ number n is not composite (is prime)
- practical usefulness of randomized algorithms: repeating the algorithm several times dramatically increases the likelihood of being correct
 - if I get only NO’s in m attempts then with probability of error $\leq 0.5^m$ can say that n is not composite (prime)

Randomized algorithms – primality testing

- decide whether or not a number n is prime
- applications in cryptography

Algorithm 1.1.1 (Trial Division)

INPUT: Integer $n \geq 2$.

METHOD:

```
0   i: integer;  
1   i ← 2;  
2   while i · i ≤ n repeat  
3       if i divides n  
4           then return 1;  
5       i ← i + 1;  
6   return 0;
```

- $n = 74838457648748954900050464578792347604359487509026452654305481$
- n has 62 digits, $\sqrt{n}$ has 31 digits
- the basic algorithm takes 10^{13} years to output 0, n is a prime number
- efficient algorithm to primality testing?

Randomized algorithms – primality testing

- efficient algorithm to test a number being composite (not prime)?

Algorithm 1.2.1 (Lehmann's Primality Test)

INPUT: Odd integer $n \geq 3$, integer $\ell \geq 2$.

METHOD:

```
0  a, c: integer; b[1..ℓ]: array of integer;
1  for i from 1 to ℓ do
2      a ← a randomly chosen element of {1, ..., n - 1};
3      c ← a(n-1)/2 mod n;
4      if c ∉ {1, n - 1}
5          then return 1;
6          else b[i] ← c;
7  if b[1] = ... = b[ℓ] = 1
8      then return 1;
9      else return 0;
```

- the algorithm returns 1 = it takes the decision that the number is composite (not prime)
- the algorithm returns 0 = it takes the decision that the number is not composite (prime)
- repeat for ℓ times lines 2-6
 - take a random number a in $\{1, \dots, n-1\}$ and compute $c = a^{(n-1)/2} \bmod n$
 - if n is prime then c should be 1 or $n-1$ (50% - 50% if a is random)
 - $n = 7$: $1^3 \bmod 7 = 1$, $2^3 \bmod 7 = 1$, $3^3 \bmod 7 = 6$, $4^3 \bmod 7 = 1$, $5^3 \bmod 7 = 6$, $6^3 \bmod 7 = 6$
 - line 5: return 1 if c is not 1 or $n-1$, so we are sure that n is composite (not prime)
 - if among all c there is at least one $= n-1$ return 0 (not composite), otherwise return 1 (composite)
 - could have errors

Randomized algorithms – primality testing

- probability of making an error?
- case 1: n is prime (the desired output is 0)
 - 50% chance that c is 1
 - 50% chance that c is $n-1$
 - error if all $c = 1$ with probability $2^{-\ell}$
- case 2: n is composite (the desired output is 1)
 - make an error if all c are 1 and $n-1$
 - it can be shown that if there exist some number a in $\{1, \dots, n-1\}$ that satisfies $a^{(n-1)/2} \bmod n = n-1$, then more than half of the elements in $\{1, \dots, n-1\}$ satisfy $a^{(n-1)/2} \bmod n \notin \{1, n-1\}$.
 - this means that the probability that the loop in lines 1–6 runs for ℓ rounds is no more than $2^{-\ell}$
 - the probability that output 0 is produced cannot be larger than $2^{-\ell}$
 - take ℓ around 1000, the probability will be very closed to 100% for practical purposes

Algorithm 1.2.1 (Lehmann's Primality Test)

INPUT: Odd integer $n \geq 3$, integer $\ell \geq 2$.

METHOD:

```
0  a, c: integer; b[1..ℓ]: array of integer;
1  for i from 1 to ℓ do
2      a ← a randomly chosen element of {1, ..., n-1};
3      c ← a(n-1)/2 mod n;
4      if c ∉ {1, n-1}
5          then return 1;
6          else b[i] ← c;
7  if b[1] = ... = b[ℓ] = 1
8      then return 1;
9      else return 0;
```

Randomized algorithms – primality testing

- computational complexity?
- do rapid exponentiation in line 3
 - $2 \log_2 n$ multiplications and divisions of numbers less than n^2
- represent a number n on $\log_{10}(n)$ digits
- overall we have $O((\log_{10} n)^3)$ – suitable on modern computers

Algorithm 1.2.1 (Lehmann's Primality Test)

INPUT: Odd integer $n \geq 3$, integer $\ell \geq 2$.

METHOD:

```
0  a, c: integer; b[1..ℓ]: array of integer;
1  for i from 1 to ℓ do
2      a ← a randomly chosen element of {1, ..., n-1};
3      c ← a(n-1)/2 mod n;
4      if c ∉ {1, n-1}
5          then return 1;
6          else b[i] ← c;
7  if b[1] = ... = b[ℓ] = 1
8      then return 1;
9      else return 0;
```

$$P \subseteq RP \subseteq NP$$

- P = class of decision problems that are quickly (polynomial runtime) solvable
- RP = class of decision problems that are quickly (polynomial runtime) solvable by randomized algorithms
- NP = class of decision problems that are quickly checkable (polynomial time),
 NP – nondeterministic polynomial time (Sudoku)

We have that:

- $P \subseteq RP$, as we use the same algorithm with polynomial time to solve a problem from P that does not use the random inputs
- $RP \subseteq NP$, as the solution of the randomized algorithm can be quickly (in polynomial time) checkable
- It is believed that $P \neq NP$

$$P = RP \neq NP ?$$

- The scientific community believes that the following relation holds:
$$P = RP \neq NP$$
- In 2002, formal proof that there exist a polynomial algorithm to check the primality of a number. This indirectly suggests that $P = RP$.



PRIMES is in P

Manindra Agrawal Neeraj Kayal
Nitin Saxena*

Department of Computer Science & Engineering
Indian Institute of Technology Kanpur
Kanpur-208016, INDIA
Email: {manindra,kayal,nitinsa}@iitk.ac.in

Abstract

We present an unconditional deterministic polynomial-time algorithm that determines whether an input number is prime or composite.

Relation between consistency and PAC learning

Theorem

Let $\mathcal{H}$ be a hypothesis class that is efficient PAC learnable. Then, there exists a randomized algorithm which solves the problem of finding a hypothesis in $\mathcal{H}$ consistent with a given training sample, and which has runtime polynomial in m (the length of the training sample).

Proof

Let S^* be a training sample, with $|S^*| = m^*$ and let h^* be the function that generates the labels on points in S^* . We want to find a consistent hypothesis with S^* .

Consider the following probability distribution over domain $\mathcal{X}$:

$$\mathcal{D}(x) = \begin{cases} 1/m^*, & \text{if } x \text{ occurs in } S^* \\ 0, & \text{otherwise} \end{cases}$$

Relation between consistency and PAC learning

$\mathcal{H}$ is efficient PAC learnable, so there exists a learning algorithm A such that:

given $\varepsilon = 1/(m^*+1)$, $\delta = 1/2$, $f = h^*$, *distribution* $\mathcal{D}$ over $\mathcal{X}$ when we run the learning algorithm A on a training set S , consisting of $m \geq m_{\mathcal{H}}(1/(m^*+1), 1/2)$ examples sampled i.i.d. from $\mathcal{D}$ and labeled by h^* the algorithm A returns a hypothesis $h_S \in \mathcal{H}$ such that, with probability at least $1-1/2 = 1/2$ (over the choice of examples), $L_{\mathcal{D}, h^*}(h_S) \leq 1/(m^*+1)$.

We want to design a randomized algorithm A^* for finding a hypothesis which agrees (training error = 0) with the given training sample S^* . We do the following:

- compute $m = m_{\mathcal{H}}(1/(m^*+1), 1/2)$
- construct a training sample S , $|S| = m$ with samples i.i.d from $\mathcal{D}$
- run the given PAC learning algorithm A on S
- check if the hypothesis $A(S) = h_S$ agrees with S^* (the probability $> 1/2$)
- if the hypothesis h_S does not agree with S^* return NO. Else return h_S . h_S will make 0 error (based on $\mathcal{D}$) so is consistent.

A^* will succeed with probability greater than $1/2$. If A is polynomial then also A^* is polynomial. Repeat A^* for $l = 1000$ times. The chance not to succeed is $\leq 0.5^{1000}$

Do we always have efficient PAC learning?

Theorem

Let $\mathcal{H}$ be a hypothesis class that is efficient PAC learnable. Then, there exists a randomized algorithm which solves the problem of finding a hypothesis in $\mathcal{H}$ consistent with a given training sample, and which has runtime polynomial in m (the length of the training sample).

We will show that there exist classes that are PAC learnable (in the realizable case) for which the sample complexity is polynomial but the runtime is not polynomial:

- learning sometimes is intractable
- “proper” learning vs “improper” learning

Intractability of learning

Learning 3-Term DNF

Consider $\mathcal{H}_{3\text{DNF}}^n$ = class of 3-term disjunctive normal form formulae consisting of hypothesis of the form $h: \{0,1\}^n \rightarrow \{0,1\}$,

$$h(\mathbf{x}) = A_1(\mathbf{x}) \vee A_2(\mathbf{x}) \vee A_3(\mathbf{x}),$$

where $A_1(\mathbf{x})$ is a Boolean conjunction (in $\mathcal{H}_{\text{conj}}^n$) with at most n Boolean literals $x_1, \dots, x_n$.

The output of $h(\mathbf{x})$ is 1 if either $A_1(\mathbf{x})$ or $A_2(\mathbf{x})$ or $A_3(\mathbf{x})$ outputs the label 1. If all three conjunctions output the label 0 then $h(\mathbf{x}) = 0$.

$|\mathcal{H}_{3\text{DNF}}^n| = 3^{3n} < \infty$, so is PAC learnable with sample complexity $3n \log(3/\delta)/\epsilon$ (polynomial in $1/\epsilon$, $1/\delta$, n).

However, from the computational perspective, this learning problem is hard. There is no polynomial time algorithm (unless $\text{RP} = \text{NP}$) that properly learns from training data. So ERM_H is not efficient.

We will show that if $\mathcal{H}_{3\text{DNF}}^n$ is efficient PAC learnable then the problem of graph 3-coloring problem (which is shown to be NP-complete) is in RP.

Learning 3-Term DNF

Theorem

Unless NP has randomized polynomial time algorithms, there is no proper efficient PAC-learning algorithm for the class $\mathcal{H}_{3\text{DNF}}^n$.

Proof idea

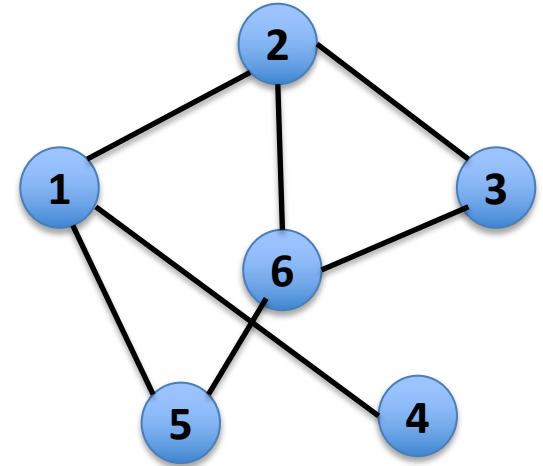
We consider that we can efficiently PAC learn the class $\mathcal{H}_{3\text{DNF}}^n$. From the preceding theorem there exist a randomized algorithm A^* that can decide consistency of 3-term DNF.

We show that the 3-term DNF problem can be reduced to the problem of the graph 3-coloring problem (which is NP-complete). So there exist an algorithm in RP that solves a problem in NP. So $\text{RP} = \text{NP}$, which we consider to be false, as we believe that $\text{P} = \text{RP} \neq \text{NP}$.

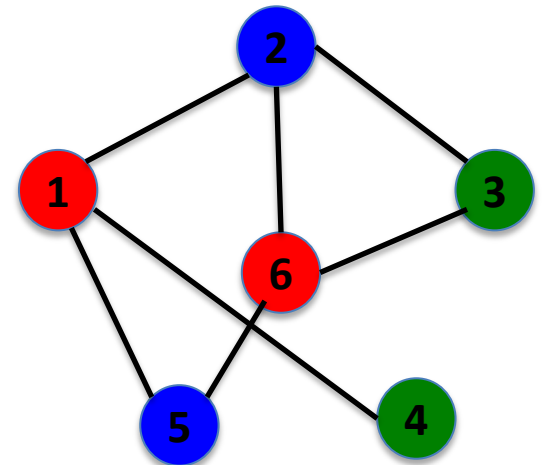
The graph 3-coloring problem

The graph 3-coloring problem

Given as input an undirected graph $G = (V, E)$ with $V = \{1, 2, \dots, n\}$ and edge set $E \subseteq V \times V$, determine if there is an assignment of a color to each element of V such that at most 3 different colors are used, and for every edge $(u, v) \in E$, vertex u and vertex v are assigned different colors.



$c : V \rightarrow \{R, G, B\}$ is a proper coloring function if for every $(u, v) \in E$ we have that $c(u) \neq c(v)$

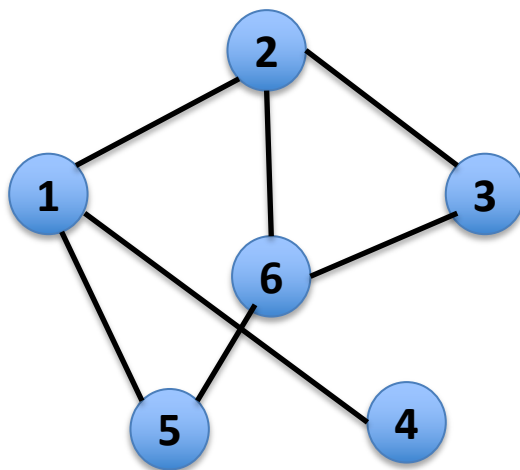


Reduction of graph 3-coloring problem to the 3-term DNF problem

We reduce an instance $G = (V, E)$ to a labeled set S_G of examples (of n bits) consisting of $|V|$ positive examples (S_G^+) and $|E|$ negative examples (S_G^-).

Each vertex i is encoded as the vector with 0 on position i and 1 on all other positions (0-hot encoding) and has label +1: $((1, 1, \dots, 1, 0, 1, \dots, 1), +1)$

Each edge $(i, j) \in E$ is encoded as the vector with 0 on positions i and j and 1 on all other positions and has label 0: $((1, \dots, 1, 0, 1, \dots, 1, 0, 1, \dots, 1), 0)$



S_G^+

$((0, 1, 1, 1, 1, 1), 1)$
 $((1, 0, 1, 1, 1, 1), 1)$
 $((1, 1, 0, 1, 1, 1), 1)$
 $((1, 1, 1, 0, 1, 1), 1)$
 $((1, 1, 1, 1, 0, 1), 1)$
 $((1, 1, 1, 1, 1, 0), 1)$

S_G^-

$((0, 0, 1, 1, 1, 1), 0)$
 $((0, 1, 1, 0, 1, 1), 0)$
 $((0, 1, 1, 1, 0, 1), 0)$
 $((1, 0, 0, 1, 1, 1), 0)$
 $((1, 0, 1, 1, 1, 0), 0)$
 $((1, 1, 0, 1, 1, 0), 0)$
 $((1, 1, 1, 1, 0, 0), 0)$

Reduction of graph 3-coloring problem to the 3-term DNF problem

The graph G is 3-colorable $\Leftrightarrow S_G = S_G^+ \cup S_G^-$ is consistent with some 3-term DNF formula.

Proof ‘ $\Rightarrow$ ’ Suppose graph G is 3-colorable. Fix such a coloring.

Let R = set of all vertices with color red (= $\{1,6\}$ in figure)

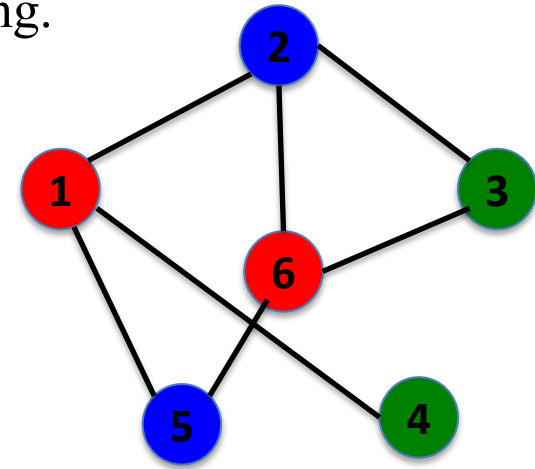
Let T_R the conjunction of all variables $x_1, x_2, \dots, x_n$ whose index does not appear in R . ($T_R = x_2 \wedge x_3 \wedge x_4 \wedge x_5$)

Then, the positive example corresponding to a vertex i in R satisfies T_R because x_i does not appear in T_R .

Every negative example corresponding to an edge (i, j) does not satisfy T_R , as either x_i or x_j appear in T_R .

Do the same reasoning for colors green and blue and obtain T_B, T_G .

Take the 3-term DNF formula as $T_R \vee T_B \vee T_G$.



Reduction of graph 3-coloring problem to the 3-term DNF problem

The graph G is 3-colorable $\Leftrightarrow S_G = S_G^+ \cup S_G^-$ is consistent with some 3-term DNF formula.

Proof ‘ $\Leftarrow$ ’ Suppose that the formula $T_R \vee T_B \vee T_G$ is consistent with S_G .

Define a coloring of G as follows: the color of vertex i is red/green/blue if the positive example corresponding to the vertex i satisfies $T_R/T_G/T_B$ (choose random color for ties).

If $i \neq j$ have the same color (let's say red) then:

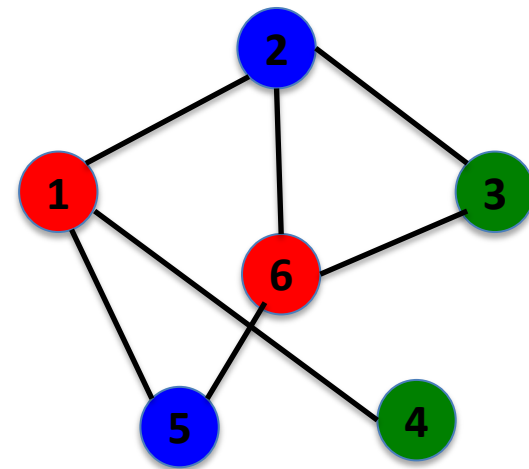
$T_R(1, \dots, 1, 0, 1, \dots, 1, 1, 1, \dots, 1) = 1$ (0 on position i)

$T_R(1, \dots, 1, 1, 1, \dots, 1, 0, 1, \dots, 1) = 1$ (0 on position j)

So literals $x_i, \bar{x}_i, x_j, \bar{x}_j$ and $\bar{x}_k$ ($k \neq i, j$) cannot appear in T_R

So we have that:

$T_R(1, \dots, 1, 0, 1, \dots, 1, 0, 1, \dots, 1) = 1$. So $(i, j) \notin E$.



Main result

The graph G is 3-colorable $\Leftrightarrow S_G = S_G^+ \cup S_G^-$ is consistent with some 3-term DNF formula.

So, the class $\mathcal{H}_{3\text{DNF}}^n$ is not efficient PAC learnable under the assumption that NP-complete problems cannot be solved with high probability by a probabilistic polynomial-time algorithm (technically, under the assumption $\text{RP} \neq \text{NP}$).

The technique can be used to generalize the result for $\mathcal{H}_{k\text{DNF}}^n$ (k -term DNF formulae), where $k \geq 3$.

“Consistent learning is hard” $\nRightarrow$ “learning is hard”

“Consistent learning is hard” $\Rightarrow$ “*proper* learning is hard”

A proper learning algorithm is a learning algorithm that must output $h \in \mathcal{H}$

Improper learning

Improper learning of 3-term DNFs

Use the distribution rule to obtain:

$$(a \wedge b) \vee (c \wedge d) = (a \vee c) \wedge (a \vee d) \wedge (b \vee c) \wedge (b \vee d)$$

Apply for the 3-term DNF formula to obtain a 3-CNF formulae:

$$A_1 \vee A_2 \vee A_3 = \bigwedge_{u \in A_1, v \in A_2, w \in A_3} (u \vee v \vee w) = \bigwedge_{u \in A_1, v \in A_2, w \in A_3} y_{u,v,w}$$

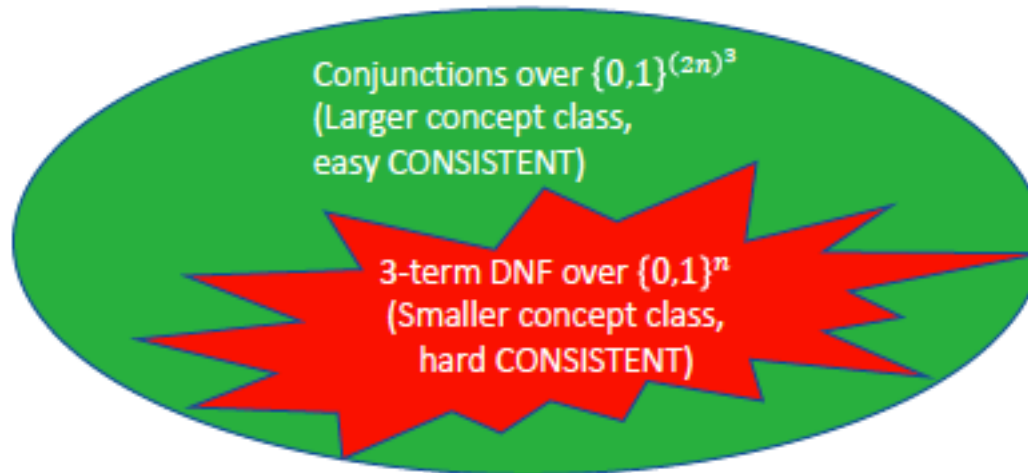
each u,v,w can take $2n$ values in $\{x_1, \bar{x}_1, x_2, \bar{x}_2, \dots, x_n, \bar{x}_n\}$

So we have that a 3-term DNF can be viewed as a conjunction of $(2n)^3$ variables:

$$H_{3DNF}^n \subseteq H_{conj}^{(2n)^3} \quad \left| H_{conj}^{(2n)^3} \right| = 3^{(2n)^3} + 1$$

We can efficiently PAC learn the new class of conjunctions, $H_{conj}^{(2n)^3}$ with sample complexity $(n^3 + \log(1/\delta))/\epsilon$. The overall runtime of this approach is polynomial in $1/\epsilon$, $1/\delta$, n . Pay polynomially in sample complexity, gain exponentially in computational complexity.

Improper learning



	3-term DNF	Conjunctions over $\{0,1\}^{(2n)^3}$
Sample complexity	$O\left(\frac{n + \log \frac{1}{\delta}}{\epsilon}\right)$	$O\left(\frac{n^3 + \log \frac{1}{\delta}}{\epsilon}\right)$ ↑
CONSISTENT Computational-Complexity	NP-Hard	$O\left(n^3 \times \frac{n^3 + \log \frac{1}{\delta}}{\epsilon}\right)$ ↓

Next time

10	Boosting	101
10.1	Weak Learnability	102
10.2	AdaBoost	105
10.3	Linear Combinations of Base Hypotheses	108
10.4	AdaBoost for Face Recognition	110
10.5	Summary	111
10.6	Bibliographic Remarks	111
10.7	Exercises	112